

ĐẠI HỌC BÁCH KHOA HÀ NỘI
KHOA TOÁN - TIN

ĐỒ ÁN TỐT NGHIỆP

Xây dựng Chatbot hỏi đáp quy chế sinh viên

LÊ QUANG ĐỨC

Email: duclq227221@sis.hust.edu.vn

Mã sinh viên: 20227221

Chuyên ngành Hệ thống Thông tin Quản lý

Giảng viên hướng dẫn : TS. Ngô Thị Hiền

Chữ ký GVHD

HÀ NỘI, 3/2026

NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN

1. Mục tiêu và nội dung của đề án

(a) Mục tiêu:

.....
.....
.....
.....

(b) Nội dung:

.....
.....
.....
.....

2. Kết quả đạt được:

.....
.....
.....
.....
.....

3. Ý thức làm việc của sinh viên:

.....
.....
.....
.....
.....

Hà Nội, ngày tháng năm 2025

Giảng viên hướng dẫn

TS. Ngô Thị Hiền

Mục lục

BẢNG KÝ HIỆU VÀ CHỮ VIẾT TẮT	7
MỞ ĐẦU	8
1 Cơ sở lý thuyết	8
1.1 Tổng quan về mô hình ngôn ngữ lớn	8
1.1.1 Kiến trúc Transformer và nguyên lý hoạt động	8
1.1.2 Ưu điểm và thách thức	11
1.2 Retrieval-Augmented Generation (RAG) truyền thống	12
1.2.1 Khái niệm	12
1.2.2 Hạn chế	13
1.3 Agentic RAG	14
1.3.1 Khái niệm	14
1.3.2 Sự khác biệt giữa RAG và Agentic RAG	15
1.4 Cross-Lingual Information Retrieval (CLIR)	16
1.4.1 Khái niệm	16
1.4.2 Ứng dụng trong chatbot đa ngôn ngữ	16
1.5 Tech stack và các kỹ thuật sử dụng	18
1.5.1 Tech stack	18
1.5.2 Các kỹ thuật sử dụng	19
2 Thiết kế hệ thống AgenticRAG	20
2.1 Kiến trúc tổng thể hệ thống	20
2.1.1 Sơ đồ kiến trúc tổng thể	20
2.1.2 Các thành phần chính	20
2.2 Thu thập và xử lý dữ liệu	20
2.3 Thiết kế Agent và Reasoning Loop	20
2.4 Thiết kế hỗ trợ đa ngôn ngữ (CLIR)	20
3 Triển khai và đánh giá	21
3.1 Xây dựng cơ sở tri thức	21
3.2 Triển khai thực tế	21
3.3 Kịch bản thử nghiệm	21
3.4 Phương pháp đánh giá	21
3.5 Phân tích kết quả	21
KẾT LUẬN	22
TÀI LIỆU THAM KHẢO	23

Danh sách hình vẽ

1.1	Kiến trúc mô hình Transformer [4]	9
1.2	Kiến trúc RAG [1]	13
1.3	Kiến trúc Agentic RAG [3]	14
1.4	Biểu diễn vectơ từ phân tán [2]	16
1.5	Quy trình xử lý truy vấn đa ngôn ngữ trong hệ thống chatbot	17

Danh sách bảng

1.1 So sánh đặc điểm giữa RAG Truyền thống và Agentic RAG 15

BẢNG KÝ HIỆU VÀ CHỮ VIẾT TẮT

RAG	Retrieval-Augmented Generation
LLM	Large Language Model (Mô hình ngôn ngữ lớn)
LoRA	Low-Rank Adaptation
QLoRA	Quantized Low-Rank Adaptation
NLP	Natural Language Processing
FAQ	Frequently Asked Questions (Các câu hỏi thường gặp)
BERTScore	BERT-based Evaluation Score
RNN	Recurrent Neural Network (Mạng nơ-ron hồi quy)
LSTM	Long Short-Term Memory (Bộ nhớ dài-ngắn Hạn)

Chương 1

Cơ sở lý thuyết

1.1 Tổng quan về mô hình ngôn ngữ lớn

Trong những năm gần đây, lĩnh vực xử lý ngôn ngữ tự nhiên (NLP) đã chứng kiến một bước chuyển mình mang tính cách mạng với sự ra đời của các mô hình ngôn ngữ lớn (LLMs). Đây là các mô hình học sâu (Deep Learning) được huấn luyện trên một lượng dữ liệu văn bản khổng lồ, sở hữu hàng tỷ, thậm chí hàng nghìn tỷ tham số. Thay vì được huấn luyện cho một tác vụ cụ thể, LLMs được huấn luyện để dự đoán từ tiếp theo trong một chuỗi, một nhiệm vụ tưởng chừng đơn giản nhưng lại giúp mô hình học được các cấu trúc ngữ pháp, ngữ nghĩa, và một lượng lớn kiến thức về thế giới.

Sự phát triển của LLMs đã mở ra những khả năng vượt bậc trong việc hiểu và tạo sinh ngôn ngữ tự nhiên, cho phép máy tính thực hiện các tác vụ phức tạp như trả lời câu hỏi, tóm tắt văn bản, dịch thuật, và thậm chí là lập trình, với độ chính xác và mạch lạc đáng kinh ngạc. Nền tảng kiến trúc cốt lõi đằng sau sự thành công của hầu hết các LLM hiện đại chính là kiến trúc Transformer.

1.1.1 Kiến trúc Transformer và nguyên lý hoạt động

Trước năm 2017, các mô hình NLP hàng đầu chủ yếu dựa vào các kiến trúc tuần tự như mạng nơ-ron hồi quy (RNN) hay mạng bộ nhớ dài-ngắn hạn (LSTM). Mặc dù hiệu quả, các mô hình này gặp hai hạn chế lớn:

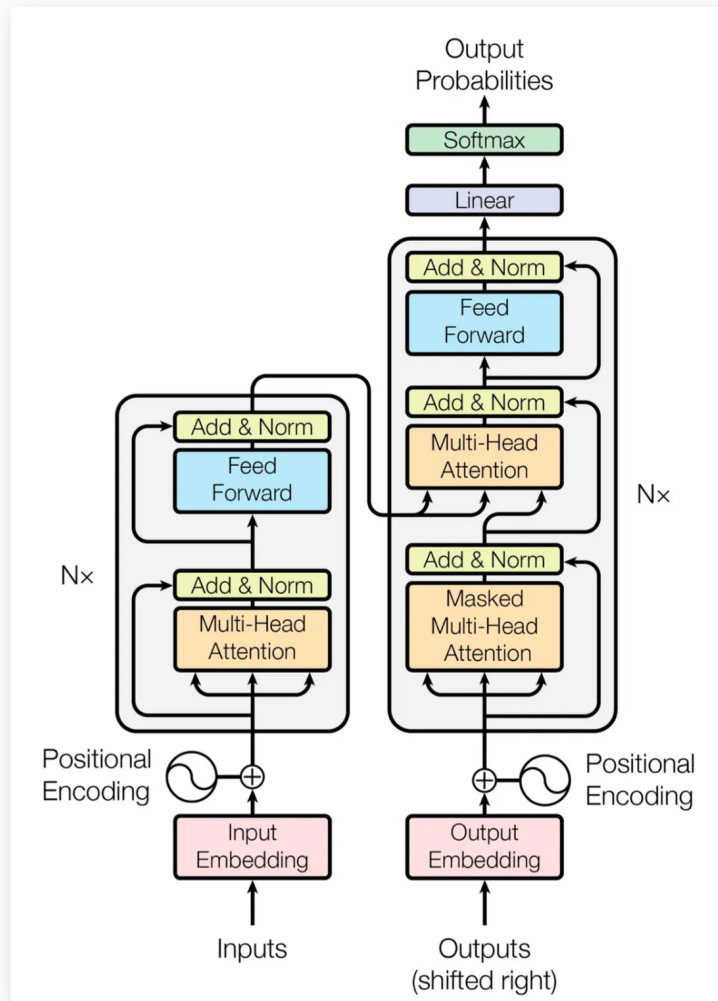
1. Khó khăn trong việc nắm bắt các phụ thuộc ngữ cảnh ở xa (long-range dependencies) do phải xử lý văn bản theo từng từ.
2. Khó khăn trong việc song song hóa quá trình huấn luyện.

Sự ra đời của kiến trúc Transformer trong bài báo "Attention Is All You Need" (Vaswani et al., 2017) đã giải quyết triệt để cả hai vấn đề này. Transformer loại bỏ hoàn toàn cơ chế hồi quy và chỉ dựa vào một cơ chế duy nhất gọi là **Self-Attention (Tự chú ý)**.

Transformer tuân theo kiến trúc tổng thể encoder-decoder.

1. **Encoder (Bộ mã hóa):** Nằm ở nửa bên trái của 1.1, bộ mã hóa có nhiệm vụ ánh xạ một chuỗi đầu vào gồm các biểu diễn ký hiệu $(x_1, \dots, x_n)$ thành một chuỗi các biểu diễn liên tục $z = (z_1, \dots, z_n)$.
2. **Decoder (Bộ giải mã):** Nằm ở nửa bên phải của 1.1, bộ giải mã nhận đầu ra z từ encoder và tạo ra một chuỗi đầu ra $(y_1, \dots, y_m)$, từng phần tử một. Mô hình này có tính

chất tự hồi quy (auto-regressive), nghĩa là nó tiêu thụ các ký hiệu đã tạo ra trước đó làm đầu vào bổ sung khi tạo ra ký hiệu tiếp theo.



Hình 1.1: Kiến trúc mô hình Transformer [4]

Cả bộ mã hóa và giải mã đều được cấu tạo từ các khối (stacks) gồm $N = 6$ tầng (layers) giống hệt nhau.

Tầng Encoder

Mỗi tầng trong bộ mã hóa có hai tầng phụ (sub-layers):

1. **Multi-Head Self-Attention:** Đây là nơi chuỗi đầu vào "tự chú ý" đến chính nó. Nó giúp mô hình xác định xem mỗi từ trong câu nên "chú ý" đến những từ nào khác trong cùng câu đó để hiểu rõ ngữ cảnh.
2. **Position-wise Feed-Forward Network:** Một mạng nơ-ron truyền thẳng (FFN) 2 lớp đơn giản.

Một điểm cực kỳ quan trọng là sau mỗi lớp con, mô hình đều áp dụng một kết nối phần dư (residual connection) và chuẩn hóa tầng (layer normalization). Tức là đầu ra sẽ là

$LayerNorm(x + Sublayer(x))$. Việc này giúp huấn luyện các mô hình mà không bị hiện tượng "vanishing gradients" (gradient triệt tiêu).

Tầng Decoder

Mỗi tầng trong bộ giải mã cũng bao gồm $N = 6$ tầng giống hệt nhau. Ngoài hai tầng phụ như trong mỗi tầng encoder, bộ giải mã chèn thêm một tầng phụ thứ ba:

1. **Masked Multi-Head Self-Attention:** Tầng này được sửa đổi để ngăn các vị trí chú ý đến các vị trí tiếp theo. Việc che giấu (masking) này đảm bảo rằng dự đoán cho vị trí i chỉ có thể phụ thuộc vào các đầu ra đã biết ở các vị trí trước i , bảo toàn tính chất tự hồi quy.
2. **Multi-Head Attention (Encoder-Decoder):** Tầng phụ thứ hai thực hiện cơ chế chú ý đa đầu (multi-head attention) trên đầu ra của khối encoder. Điều này cho phép mọi vị trí trong decoder chú ý đến tất cả các vị trí trong chuỗi đầu vào.
3. **Mạng Feed-Forward (Position-wise):** Tương tự như trong encoder.

Tương tự Encoder, mỗi lớp con này đều có kết nối phần dư và chuẩn hóa tầng.

Scaled Dot-Product Attention

Một hàm chú ý có thể được mô tả là ánh xạ một truy vấn (Q) và một tập hợp các cặp khóa-giá trị ($K - V$) tới một đầu ra. Đầu ra được tính bằng tổng có trọng số của các giá trị (V), trong đó trọng số gán cho mỗi giá trị được tính bằng hàm tương thích giữa truy vấn (Q) với khóa (K) tương ứng.

Cơ chế chú ý này được gọi là "Scaled Dot-Product Attention". Đầu vào bao gồm các truy vấn (Q) và khóa (K) có chiều d_k , và các giá trị (V) có chiều d_v . Cơ chế này tính toán tích vô hướng của truy vấn với tất cả các khóa, chia mỗi kết quả cho $\sqrt{d_k}$, và sau đó áp dụng hàm softmax để thu được trọng số cho các giá trị.

Công thức tính toán attention là:

$$Attention(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (1.1)$$

Việc chia cho $\sqrt{d_k}$ là một yếu tố tỷ lệ (scaling factor). Khi d_k lớn, tích vô hướng có thể trở nên rất lớn, đẩy hàm softmax vào vùng có gradient cực nhỏ. Việc chia tỷ lệ này giúp chống lại hiện tượng đó.

Multi-Head Attention

Thay vì thực hiện một hàm attention duy nhất với các khóa, giá trị và truy vấn có kích thước d_{model} , Transformer thấy lợi ích khi chiếu tuyến tính các truy vấn, khóa và giá trị h lần bằng các phép chiếu tuyến tính được học khác nhau.

1. **Phép Chiếu:** Các truy vấn, khóa và giá trị được chiếu tuyến tính h lần tới các chiều d_k, d_k, d_v tương ứng.

⁰Nội dung tham khảo từ bài báo "Attention Is All You Need" (Vaswani et al., 2017)[4].

2. **Thực hiện song song:** Hàm attention được thực hiện song song trên mỗi phiên bản được chiếu này, tạo ra h đầu ra d_v -chiều (gọi là "heads").
3. **Kết hợp:** Các đầu ra này được nối lại và sau đó được chiếu tuyến tính một lần nữa để tạo ra giá trị cuối cùng.

Multi-head attention cho phép mô hình cùng lúc chú ý đến thông tin từ các không gian biểu diễn khác nhau tại các vị trí khác nhau. Nếu chỉ dùng một đầu chú ý việc lấy trung bình sẽ cản trở khả năng này.

Transformer sử dụng multi-head attention theo ba cách:

1. **Encoder-Decoder Attention:** Truy vấn (Q) đến từ tầng decoder trước đó, còn khóa (K) và giá trị (V) đến từ đầu ra của encoder.
2. **Encoder Self-Attention:** Tất cả Q, K, V đều đến từ đầu ra của tầng encoder trước đó.
3. **Decoder Self-Attention (Masked):** Tất cả Q, K, V đều đến từ đầu ra của tầng decoder trước đó, nhưng được che giấu (masking) để ngăn dòng thông tin "nhìn về tương lai" (leftward information flow).

Position-wise Feed-Forward Networks

Ngoài các tầng phụ chú ý, mỗi tầng trong encoder và decoder đều chứa một mạng nơ-ron feed-forward kết nối đầy đủ. Mạng này được áp dụng cho từng vị trí một cách riêng biệt và giống hệt nhau. Nó bao gồm hai phép biến đổi tuyến tính với một hàm kích hoạt ReLU ở giữa.

$$FFN(x) = \max(0, xW_1 + b_1)W_2 + b_2 \quad (1.2)$$

Mặc dù các phép biến đổi tuyến tính là giống nhau trên các vị trí khác nhau, chúng sử dụng các tham số khác nhau giữa các tầng.

1.1.2 Ưu điểm và thách thức

Các mô hình ngôn ngữ lớn, được xây dựng trên kiến trúc Transformer, đã mang lại một cuộc cách mạng trong lĩnh vực trí tuệ nhân tạo. Tuy nhiên, bên cạnh những lợi ích vượt trội, việc ứng dụng chúng cũng đi kèm với nhiều thách thức đáng kể.

Ưu điểm mô hình ngôn ngữ lớn:

LLMs sở hữu nhiều khả năng đột phá, khiến chúng trở nên hữu ích trong hàng loạt ứng dụng:

- **Tính đa dụng và linh hoạt:** Một trong những ưu điểm lớn nhất là khả năng thực hiện nhiều tác vụ khác nhau chỉ với một mô hình duy nhất. Các tác vụ này bao gồm dịch thuật, tóm tắt văn bản, trả lời câu hỏi, sáng tạo nội dung, và thậm chí là tạo sinh mã nguồn lập trình.
- **Tự động hóa và nâng cao hiệu suất:** Tự động hóa các tác vụ lặp đi lặp lại và tốn thời gian. Điều này giúp giải phóng sức lao động của con người cho các công việc mang tính chiến lược hơn.

- **Khả năng học trong ngữ cảnh (In-context Learning):** LLMs thể hiện khả năng học hỏi nhanh chóng, đặc biệt là học theo ngữ cảnh (hay còn gọi là zero-shot và few-shot learning). Chúng có thể thực hiện các tác vụ mới mà không cần huấn luyện lại hay tinh chỉnh chuyên biệt, mà chỉ cần một vài ví dụ hoặc mô tả làm đầu vào.

Thách thức và hạn chế

Mặc dù có nhiều ưu điểm, việc triển khai và ứng dụng LLMs phải đối mặt với nhiều rào cản lớn:

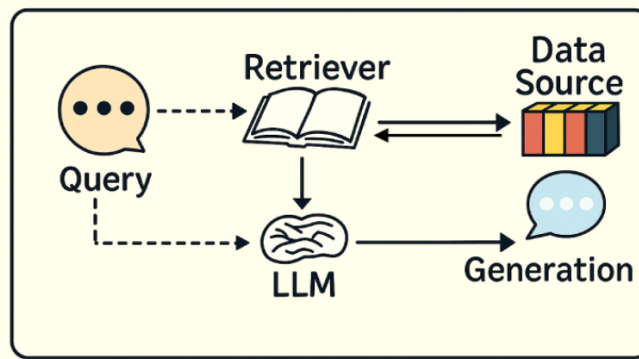
- **Hiện tượng "Ảo giác" (Hallucination):** LLMs có xu hướng tạo ra các thông tin nghe có vẻ hợp lý nhưng lại hoàn toàn sai sự thật, không dựa trên dữ liệu đầu vào hoặc không tương thích với ý định của người dùng. Điều này làm giảm đáng kể độ tin cậy của mô hình, đặc biệt trong các lĩnh vực yêu cầu tính chính xác cao như y tế hay pháp lý.
- **Chi phí tính toán và tài nguyên:** Việc huấn luyện các mô hình với hàng trăm tỷ tham số đòi hỏi nguồn tài nguyên tính toán khổng lồ, đặt ra các vấn đề về chi phí và tính bền vững của công nghệ. Quá trình triển khai và thực thi (inference) cũng rất phức tạp và tốn kém.
- **Rủi ro về an toàn và bảo mật:**
 - **Quyền riêng tư:** LLMs có thể vô tình làm rò rỉ thông tin cá nhân, nội bộ có trong dữ liệu huấn luyện hoặc trong câu lệnh của người dùng.
 - **Tấn công:** Các mô hình dễ bị tấn công bằng "prompt injection", nơi kẻ tấn công sử dụng các câu lệnh đặc biệt để lừa mô hình bỏ qua các quy tắc an toàn và tạo ra nội dung độc hại.
 - **Bản quyền:** Việc huấn luyện trên dữ liệu có bản quyền mà không có sự đồng ý đặt ra các thách thức pháp lý lớn.

1.2 Retrieval-Augmented Generation (RAG) truyền thống

1.2.1 Khái niệm

Kiến trúc Retrieval-Augmented Generation được giới thiệu như một giải pháp để giải quyết triệt để vấn đề khi tiếp cận những kiến thức chuyên biệt. Thay vì chỉ dựa vào kiến thức "nội tại" mà LLM đã học được trong quá trình huấn luyện, RAG cho phép mô hình truy cập và sử dụng kiến thức từ một nguồn dữ liệu mà ta cung cấp ngay tại thời điểm trả lời. RAG là viết tắt của từ Retrieval-Augmented Generation, là một kiến trúc trong đó mô hình ngôn ngữ được "tăng cường" bằng thông tin được "truy xuất" từ một cơ sở kiến thức bên ngoài.

What is RAG ?



Hình 1.2: Kiến trúc RAG [1]

Nguyên lý hoạt động của RAG về cơ bản có thể chia làm 2 giai đoạn:

1. Truy xuất (Retrival)

- Khi người dùng đưa ra một câu hỏi (query), hệ thống không gửi câu hỏi này trực tiếp đến LLM.
- Thay vào đó, query đầu tiên được chuyển đến một bộ truy xuất (Retriever). Bộ truy xuất này có nhiệm vụ tìm kiếm trong một cơ sở kiến thức đã được chuẩn bị trước (ví dụ: tài liệu PDF của đồ án, văn bản pháp luật, tài liệu nội bộ...).
- Kết quả của giai đoạn này là một hoặc nhiều đoạn văn bản được đánh giá là có liên quan nhất đến câu hỏi của người dùng.

2. Tăng cường và tạo sinh (Augmented Generation)

- Hệ thống lấy các đoạn văn bản (context) vừa truy xuất được và "bổ sung" chúng vào câu hỏi gốc của người dùng.
- Quá trình này thường được thực hiện thông qua một mẫu câu lệnh (prompt template) được thiết kế đặc biệt. Ví dụ:
 - Prompt cũ (chỉ có LLM): "Trình bày về ưu điểm của RAG?"
 - Prompt mới (dùng RAG): "Dựa vào thông tin sau đây: [...đoạn văn bản A được truy xuất...] và [...đoạn văn bản B được truy xuất...]. Hãy trả lời câu hỏi: Trình bày về ưu điểm của RAG?"
- Câu lệnh mới, đã được "tăng cường" ngữ cảnh này, sau đó mới được gửi đến LLM. LLM sẽ sinh ra câu trả lời dựa trên thông tin được cung cấp, thay vì phải "tưởng tượng" ra câu trả lời từ kiến thức nội tại của nó.

1.2.2 Hạn chế

Mặc dù kiến trúc Retrieval-Augmented Generation (RAG) đã cải thiện đáng kể khả năng cung cấp thông tin chính xác của mô hình ngôn ngữ lớn bằng cách kết hợp với nguồn tri thức bên ngoài, phương pháp này vẫn tồn tại nhiều hạn chế quan trọng cần được xem xét.

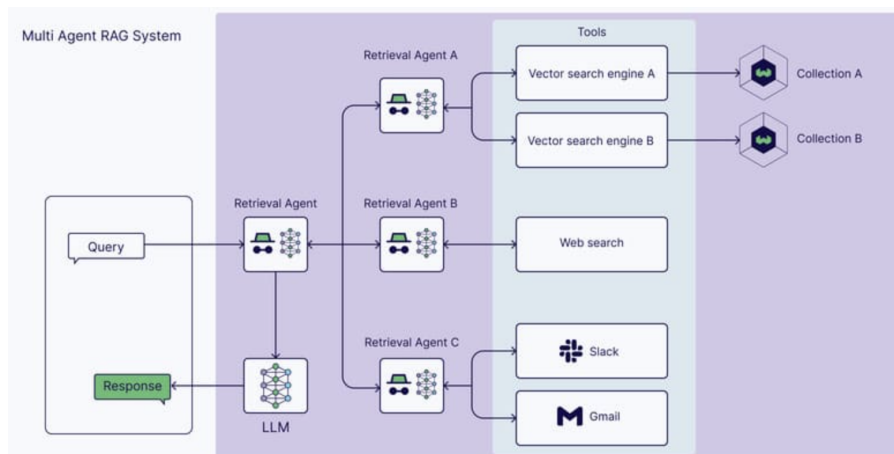
- **Thiếu tính linh hoạt:** RAG truyền thống hoạt động theo một quy trình cố định gồm hai bước là: truy xuất và tạo sinh gây thiếu khả năng tự đánh giá và điều chỉnh trong quá trình suy luận. Nếu các tài liệu được truy xuất ban đầu không đủ liên quan hoặc không chứa thông tin cần thiết, hệ thống vẫn tiếp tục sử dụng các dữ liệu đó để sinh câu trả lời mà không có cơ chế phản hồi hay cải thiện truy vấn. Điều này dẫn đến việc câu trả lời cuối cùng có thể không chính xác hoặc không đầy đủ.
- **Phụ thuộc lớn vào hiệu quả của bộ truy xuất.** Nếu người dùng nhập truy vấn một cách mơ hồ hoặc sử dụng nhiều ngôn ngữ tự nhiên phức tạp sẽ khiến cho việc ánh xạ sang không gian embedding trở nên khó khăn. Khi đó, hệ thống có thể truy xuất các đoạn văn bản không thực sự liên quan, làm giảm độ chính xác mô hình.
- **Không có khả năng Multi-hop reasoning:** Đối với các câu hỏi cần tổng hợp thông tin từ nhiều nguồn khác nhau hoặc yêu cầu liên kết giữa các đoạn kiến thức, mô hình thường gặp khó khăn do chỉ thực hiện một lần truy xuất duy nhất. Điều này hạn chế khả năng xử lý các câu hỏi phức tạp, đặc biệt trong các lĩnh vực có cấu trúc thông tin chặt chẽ như văn bản pháp lý hay quy chế.
- **Không có cơ chế kiểm chứng hoặc đánh giá lại độ tin cậy của thông tin được truy xuất.** Các đoạn văn bản được đưa vào làm ngữ cảnh có thể chứa thông tin lỗi thời, không đầy đủ hoặc không phù hợp với câu hỏi. Tuy nhiên, mô hình vẫn sử dụng các thông tin này để sinh câu trả lời, dẫn đến nguy cơ tạo ra các nội dung thiếu chính xác hoặc gây hiểu nhầm.

1.3 Agentic RAG

1.3.1 Khái niệm

Agentic RAG là một hướng tiếp cận nâng cao của kiến trúc Retrieval-Augmented Generation, trong đó hệ thống không chỉ đơn thuần thực hiện truy xuất và tạo sinh, mà còn được mở rộng với khả năng suy luận, lập kế hoạch và tự điều chỉnh thông qua cơ chế tác nhân (Agent).

Thay vì hoạt động theo một pipeline tuyến tính cố định, Agentic RAG vận hành dưới dạng một vòng lặp suy luận (reasoning loop), cho phép hệ thống đánh giá và cải thiện kết quả qua nhiều bước trung gian.



Hình 1.3: Kiến trúc Agentic RAG [3]

Bằng cách này, Agentic RAG không chỉ nâng cao độ chính xác của phần thông tin được truy xuất mà còn tối ưu hóa cách thông tin được sử dụng trong quá trình tạo nội dung. Phương pháp này đặc biệt hữu ích cho các bài toán phức tạp đòi hỏi vừa hiểu dữ liệu, vừa lập kế hoạch, suy luận nhiều bước.

Một số AI Agent trong Agentic RAG có thể kể đến như:

- **Routing Agents:** Chịu trách nhiệm xác định nguồn dữ liệu và công cụ phù hợp để xử lý truy vấn người dùng quyết định nguồn truy xuất phù hợp nhất và chọn pipeline tối ưu để tạo phản hồi.
- **Query Planning Agents:** Đóng vai trò như người điều phối, các agent này chia truy vấn phức tạp thành các bước nhỏ hơn và phân phối cho các agent khác xử lý. Sau đó, chúng tổng hợp các kết quả thu được để tạo ra phản hồi hoàn chỉnh. Cơ chế này – còn gọi là AI orchestration – đặc biệt hiệu quả với các truy vấn đa chiều hoặc có logic phân nhánh.
- **ReAct Agents (Reasoning + Acting):** Đây là một framework giúp các agent vừa lập luận, vừa hành động theo từng bước. ReAct Agents có thể xác định công cụ hoặc bước xử lý phù hợp dựa trên kết quả của các bước trước, từ đó xây dựng một chuỗi phản ứng linh hoạt và thích ứng tốt với thông tin mới.

1.3.2 Sự khác biệt giữa RAG và Agentic RAG

Sự khác biệt cốt lõi giữa RAG truyền thống và Agentic RAG nằm ở cách thức xử lý truy vấn và mức độ “thông minh” trong quá trình suy luận.

Đặc điểm	RAG Truyền thống	Agentic RAG
Quy trình	Tuyến tính (Truy xuất → Tạo sinh). Chỉ thực hiện một lần duy nhất.	Vòng lặp suy luận: Có khả năng đánh giá kết quả trung gian và thực hiện lại nếu cần.
Xử lý truy vấn	Phụ thuộc hoàn toàn vào câu hỏi ban đầu của người dùng.	Tối ưu hóa truy vấn: Có khả năng tự viết lại câu hỏi (Rewriter) để tìm kiếm hiệu quả hơn.
Kiểm soát chất lượng	Chấp nhận tất cả tài liệu truy xuất được, dễ bị nhiễu thông tin.	Cơ chế đánh giá (Grading): Lọc bỏ tài liệu kém chất lượng, chỉ giữ lại thông tin hữu ích.
Khả năng suy luận	Hạn chế với các câu hỏi phức tạp cần nhiều bước giải quyết.	Đa bước (Multi-step): Truy xuất và tổng hợp từ nhiều nguồn để trả lời các vấn đề phức tạp.
Kiến trúc	Đơn giản, dễ triển khai nhưng thiếu linh hoạt.	Phức tạp (dạng đồ thị tác nhân), nhưng linh hoạt và dễ mở rộng trong thực tế.

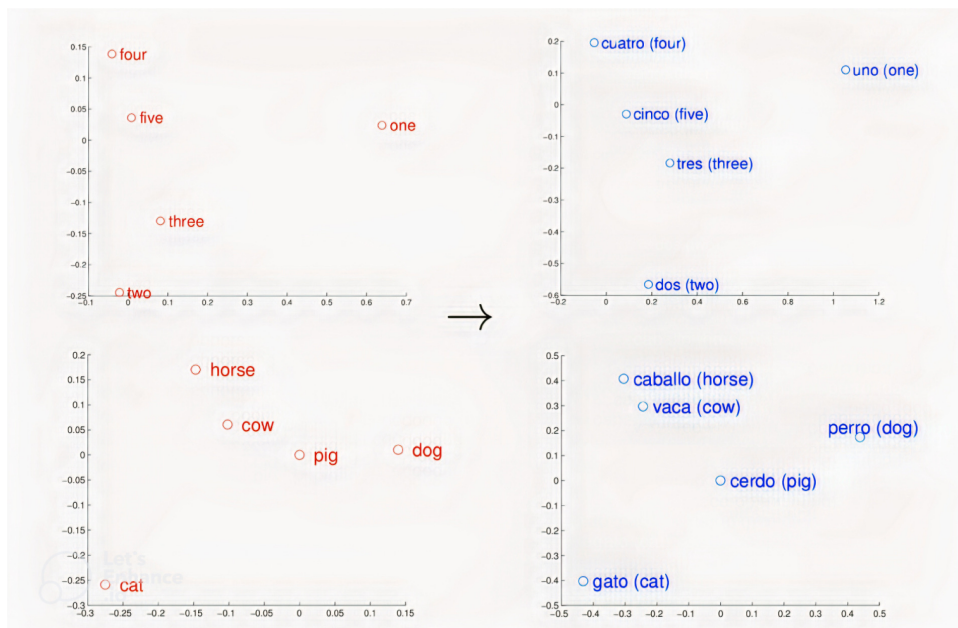
Bảng 1.1: So sánh đặc điểm giữa RAG Truyền thống và Agentic RAG

Tóm lại, Agentic RAG có thể được xem là một phiên bản nâng cao của RAG truyền thống, giúp khắc phục các hạn chế về truy xuất, suy luận và độ tin cậy. Việc áp dụng Agentic RAG trong bài toán chatbot hỏi đáp quy chế sinh viên hứa hẹn sẽ mang lại hiệu quả cao hơn trong việc cung cấp thông tin chính xác, đầy đủ và phù hợp với nhu cầu của người dùng.

1.4 Cross-Lingual Information Retrieval (CLIR)

1.4.1 Khái niệm

Cross-Lingual Information Retrieval (CLIR) là một nhánh của lĩnh vực truy xuất thông tin, cho phép hệ thống tìm kiếm và truy xuất dữ liệu khi truy vấn và tài liệu thuộc các ngôn ngữ khác nhau. Nói cách khác, CLIR hỗ trợ người dùng đặt câu hỏi bằng một ngôn ngữ (ví dụ: tiếng Anh) nhưng hệ thống vẫn có thể tìm kiếm và khai thác thông tin từ kho dữ liệu được lưu trữ bằng một ngôn ngữ khác (ví dụ: tiếng Việt).



Hình 1.4: Biểu diễn vectơ từ phân tán [2]

Có hai hướng tiếp cận phổ biến trong CLIR.

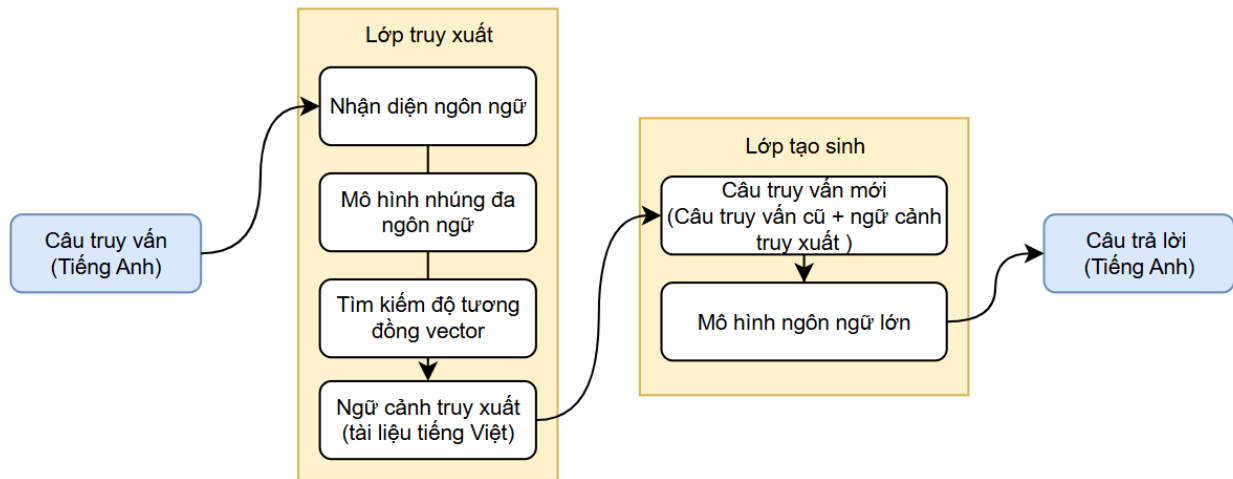
- **Phương pháp dịch truy vấn (translate-then-retrieve):** Câu hỏi của người dùng được dịch sang ngôn ngữ của tài liệu trước khi thực hiện truy xuất
- **Sử dụng mô hình embedding đa ngôn ngữ (multilingual embedding):** Ánh xạ văn bản từ nhiều ngôn ngữ khác nhau vào cùng một không gian biểu diễn. Trong cách tiếp cận này, các câu hỏi và tài liệu dù ở ngôn ngữ khác nhau vẫn có thể được so sánh trực tiếp về mặt ngữ nghĩa.

1.4.2 Ứng dụng trong chatbot đa ngôn ngữ

Trong hệ thống chatbot hỏi đáp quy chế sinh viên, dữ liệu đầu vào chủ yếu được xây dựng bằng tiếng Việt. Tuy nhiên, trong thực tế, người dùng có thể có nhu cầu đặt câu hỏi bằng tiếng

Anh, đặc biệt trong môi trường giáo dục quốc tế hoặc đối với sinh viên nước ngoài. Việc tích hợp CLIR giúp hệ thống có thể xử lý linh hoạt các truy vấn đa ngôn ngữ mà không cần xây dựng nhiều phiên bản dữ liệu riêng biệt.

Khi người dùng nhập câu hỏi bằng tiếng Anh, hệ thống sử dụng embedding đa ngôn ngữ để ánh xạ câu hỏi này vào cùng không gian với dữ liệu tiếng Việt trong cơ sở tri thức. Sau đó, quá trình truy xuất vẫn được thực hiện bình thường như với truy vấn tiếng Việt. Kết quả truy xuất sẽ là các đoạn văn bản tiếng Việt có liên quan, và mô hình ngôn ngữ lớn sẽ sử dụng các thông tin này để tạo ra câu trả lời phù hợp, có thể bằng tiếng Anh hoặc tiếng Việt tùy theo yêu cầu.



Hình 1.5: Quy trình xử lý truy vấn đa ngôn ngữ trong hệ thống chatbot

Ta xét ví dụ sau:

Câu hỏi đầu vào (tiếng Anh): "What are the conditions for academic probation?"

Cơ sở tri thức (tiếng Việt): "Sinh viên sẽ bị cảnh báo học tập nếu điểm trung bình học kỳ dưới mức quy định hoặc không đạt số tín chỉ yêu cầu."

Quy trình xử lý của hệ thống được thực hiện qua các bước sau:

- **Bước 1: Nhận diện ngôn ngữ đầu vào**
Hệ thống xác định câu hỏi được nhập bằng tiếng Anh.
- **Bước 2: Biểu diễn embedding đa ngôn ngữ**
Câu hỏi tiếng Anh được đưa qua mô hình embedding để chuyển thành vector. Đồng thời, các đoạn văn bản tiếng Việt trong cơ sở dữ liệu cũng đã được embedding từ trước. Nhờ embedding đa ngôn ngữ, cả tiếng Anh và tiếng Việt được đưa về cùng một không gian ngữ nghĩa.
- **Bước 3: Truy xuất thông tin (Retrieval)**
Hệ thống thực hiện tìm kiếm similarity trong vector database.
- **Bước 4: Tạo câu trả lời (Generation)**
Mô hình ngôn ngữ lớn (LLM) nhận đầu vào bao gồm câu hỏi gốc (tiếng Anh) và ngữ cảnh được truy xuất (tiếng Việt). Dựa trên thông tin này, mô hình sinh ra câu trả lời phù hợp.

Ví dụ đầu ra (tiếng Anh):

"Students will be placed on academic probation if their semester GPA falls below the required threshold or they fail to meet the minimum credit requirements."

1.5 Tech stack và các kỹ thuật sử dụng

1.5.1 Tech stack

Python

Đây là ngôn ngữ phổ biến trong lĩnh vực trí tuệ nhân tạo và khoa học dữ liệu, với hệ sinh thái thư viện phong phú như xử lý văn bản, xây dựng mô hình học máy và triển khai hệ thống. Python cũng hỗ trợ tốt cho việc tích hợp với các framework hiện đại trong xây dựng hệ thống RAG và Agent.

LangGraph

Công cụ này cho phép mô hình hóa luồng xử lý dưới dạng đồ thị, trong đó mỗi node đại diện cho một bước xử lý như truy xuất, đánh giá hoặc sinh câu trả lời. LangGraph hỗ trợ xây dựng các vòng lặp suy luận (reasoning loop) một cách rõ ràng và linh hoạt, giúp hệ thống có khả năng tự điều chỉnh trong quá trình xử lý truy vấn.

vLLM

Một framework tối ưu hóa cho quá trình suy luận (inference). vLLM giúp tăng tốc độ phản hồi của mô hình, đồng thời hỗ trợ xử lý nhiều yêu cầu đồng thời, phù hợp với các hệ thống chatbot triển khai thực tế. Việc sử dụng vLLM góp phần giảm độ trễ và nâng cao trải nghiệm người dùng.

Chroma DB

Chroma DB là một kho lưu trữ vector mã nguồn mở được sử dụng để lưu trữ và truy xuất các vector nhúng. Công dụng chính của nó là lưu các vector nhúng cùng với siêu dữ liệu để các mô hình ngôn ngữ lớn sử dụng sau này. Ngoài ra, nó cũng có thể được sử dụng cho các công cụ tìm kiếm ngữ nghĩa trên dữ liệu văn bản.

Docker & Kubernetes

Công nghệ container hóa này giúp chuẩn hóa môi trường chạy, giảm thiểu xung đột phụ thuộc và dễ dàng triển khai trên nhiều nền tảng khác nhau. Bên cạnh đó, hệ thống backend có thể được xây dựng các framework như FastAPI hoặc Flask để cung cấp API cho giao diện người dùng, trong khi giao diện web giúp người dùng tương tác trực tiếp với chatbot một cách thuận tiện.

1.5.2 Các kỹ thuật sử dụng

Structure-Aware & Hierarchical Chunking

Thay vì phân tách văn bản theo độ dài cố định, phương pháp này khai thác cấu trúc logic sẵn có của tài liệu quy chế, bao gồm các cấp như Chương → Điều → Khoản → Điểm. Việc phân tách theo cấu trúc này giúp mỗi đoạn văn bản (chunk) giữ được ngữ nghĩa hoàn chỉnh của một quy định cụ thể, tránh tình trạng mất ngữ cảnh khi bị cắt ngang.

Metadata Enrichment

Mỗi đoạn văn bản sau khi được phân tách sẽ được bổ sung thêm các thông tin metadata như tiêu đề, số hiệu điều khoản và đường dẫn phân cấp (ví dụ: “Quy chế đào tạo 2024 > Chương 2 > Điều 5”). Các metadata này đóng vai trò như tín hiệu ngữ nghĩa bổ sung, giúp hệ thống hiểu rõ hơn về vị trí và ý nghĩa của đoạn văn trong toàn bộ tài liệu. Đồng thời, metadata còn hỗ trợ việc giải thích và truy vết nguồn gốc câu trả lời.

ReAct Logic (Reason + Act)

Thay vì thực hiện truy xuất một lần duy nhất, tác nhân sẽ xen kẽ giữa việc phân tích câu hỏi và thực hiện các hành động cụ thể như truy xuất dữ liệu hoặc gọi công cụ. Ví dụ, tác nhân có thể suy luận “Tôi cần tìm quy định về học phí” trước khi thực hiện truy xuất trong cơ sở tri thức. Cách tiếp cận này giúp hệ thống linh hoạt hơn trong việc xử lý các câu hỏi phức tạp. Đồng thời, nó cho phép cải thiện kết quả thông qua nhiều bước trung gian.

Hybrid Search

Kết hợp giữa tìm kiếm ngữ nghĩa (dense retrieval) và tìm kiếm từ khóa truyền thống (sparse retrieval như BM25). Tìm kiếm ngữ nghĩa giúp hệ thống hiểu được ý nghĩa tổng thể của câu hỏi, trong khi tìm kiếm từ khóa đảm bảo các thuật ngữ chuyên môn quan trọng không bị bỏ sót. Nhờ đó, hệ thống vừa đảm bảo độ bao phủ thông tin, vừa nâng cao độ chính xác của kết quả truy xuất.

Query Rewriting

Query Rewriting sử dụng mô hình ngôn ngữ lớn để viết lại câu hỏi theo cách rõ ràng, đầy đủ và phù hợp hơn với hệ thống tìm kiếm. Việc này giúp cải thiện đáng kể chất lượng truy vấn đầu vào và tăng khả năng tìm được thông tin liên quan. Ngoài ra, nó còn giúp chuẩn hóa các cách diễn đạt khác nhau của cùng một ý nghĩa.

Chương 2

Thiết kế hệ thống AgenticRAG

2.1 Kiến trúc tổng thể hệ thống

Về mặt tổng thể, hệ thống bao gồm các thành phần chính: giao diện người dùng (User Interface), hệ thống backend xử lý yêu cầu, tác nhân thông minh (Agentic RAG), cơ sở tri thức (Knowledge Base) và mô hình ngôn ngữ lớn (LLM). Các thành phần này phối hợp với nhau để thực hiện quá trình tiếp nhận, xử lý và phản hồi truy vấn của người dùng.

Giao diện người dùng đóng vai trò là điểm tương tác giữa người dùng và hệ thống, cho phép nhập câu hỏi và hiển thị câu trả lời. Backend chịu trách nhiệm điều phối luồng xử lý, bao gồm việc gọi các dịch vụ embedding, truy xuất dữ liệu và sinh câu trả lời. Tác nhân Agentic RAG là thành phần trung tâm, thực hiện các bước suy luận, lập kế hoạch và điều chỉnh truy vấn. Cơ sở tri thức lưu trữ dữ liệu dưới dạng vector, hỗ trợ truy xuất nhanh chóng. Cuối cùng, mô hình ngôn ngữ lớn đảm nhiệm việc tạo sinh câu trả lời dựa trên ngữ cảnh được cung cấp.

2.1.1 Sơ đồ kiến trúc tổng thể

2.1.2 Các thành phần chính

2.2 Thu thập và xử lý dữ liệu

2.3 Thiết kế Agent và Reasoning Loop

2.4 Thiết kế hỗ trợ đa ngôn ngữ (CLIR)

Chương 3

Triển khai và đánh giá

3.1 Xây dựng cơ sở tri thức

3.2 Triển khai thực tế

3.3 Kịch bản thử nghiệm

3.4 Phương pháp đánh giá

3.5 Phân tích kết quả

KẾT LUẬN

Tài liệu tham khảo

- [1] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. *arXiv preprint arXiv:2005.11401*, 2020.
- [2] Thapelo Sindane, Vukosi Marivate, and Abiodun Modupe. Cross-lingual embedding methods and applications: A systematic review for low-resourced scenarios. *ScienceDirect (Applied AI Letters / Preprint)*, 2025. Truy cập ngày: 20/04/2026.
- [3] Anh Tuan. Agentic rag – bước tiến tiếp theo của retrieval-augmented generation?, 2025. Truy cập ngày: 20/4/2026.
- [4] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NIPS 2017)*, Long Beach, CA, USA, 2017.

PHỤ LỤC

Phụ lục A: Mẫu dữ liệu training

Phụ lục B: Giao diện hệ thống

Phụ lục C: Kết quả chi tiết đánh giá